Decode 2026

Reproducible Data Science with Git and GitHub

Lisa Karstens, PhD

July 20 2026

Where are you zooming in from?

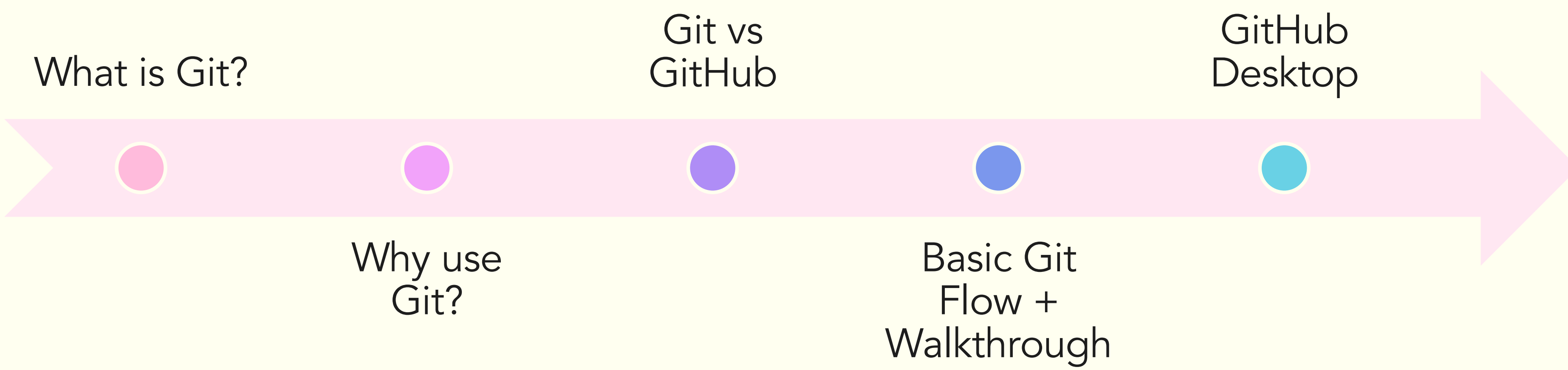
2

Drop your city/state in the chat!

BioData Sage

Karstens Lab

Today's focus



Disaster strikes when:

Your computer crashes

You change code and things break later on

You want to experiment with your code

You need to collaborate on a group project

Version control to the rescue!

Think Google Docs but for code!

Every change made to the code base is tracked
Saves snapshots of your code
Lets you easily share code with the team you are working with
Allows you to collaborate on data projects and know who contributed to the changes made
Gives you access to any previous version of your document

***Git** is a commonly used version control system*

Quick example

In our last workshop, we explored data manipulation with R using the MoTrPAC data

Key Takeaways

The Six Essential dplyr Functions:

1. `select()`: Choose which columns to keep or remove
2. `filter()`: Choose which rows to keep based on conditions
3. `arrange()`: Sort your data by one or more columns
4. `mutate()`: Create new variables or modify existing ones
5. `summarise()`: Calculate summary statistics
6. `group_by()`: Group data for group-specific operations

Quick example

In our last workshop, we explored data manipulation with R using the MoTrPAC data

Key Takeaways

The Six Essential dplyr Functions:

1. `select()`: Choose which columns to keep or remove
2. `filter()`: Choose which rows to keep based on conditions
3. `arrange()`: Sort your data by one or more columns
4. `mutate()`: Create new variables or modify existing ones
5. `summarise()`: Calculate summary statistics
6. `group_by()`: Group data for group-specific operations

What if your script breaks here and you want to go back to step 2 of your analysis?

File Name Chaos

What if you made additional
analyses and updates to your script?

- analysis.R
- analysis_v2.R
- analysis_final.R
- analysis_final_REAL.R
- analysis_final_REAL_updated.R

With Git

analysis.R

version-3

version-2

version-1

Every version is saved automatically!

Git VS GitHub

Git (local)	**GitHub** (online)
• Lives on your computer • Tracks your changes • Works offline • Personal backup	• Lives on the internet • Stores your Git repositories • Share with others • Online backup

https://github.com

Basic Git Flow

Repository/Repo

a set of files

A folder for your programming project that can contain:

1. R scripts (or any other programming language)
2. Any datasets used (e.g. CSV files, excel files)
3. Any outputs generated during your analysis
4. Any other documents related to your project

Basic Git Flow

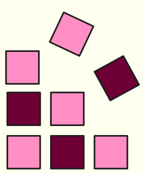
12

a set of files

Repository/Repo

Diff = add/save

- Add files to repo
- Save any new changes made to files



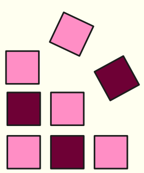
Basic Git Flow

a set of files

Repository/Repo

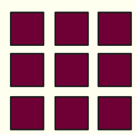
Diff = add/save

- Add files to repo
- Save any new changes made to files



Commit = describe the change

- Helps convey why the changes were made
- Confirm the changes you want to make



Basic Git Flow

a set of files

Repository/Repo

Diff = add/save

- Add files to repo
- Save any new changes made to files

Commit = describe the change

- Helps convey why the changes were made
- Confirm the changes you want to make

Push = send and store online

- Saves the new files on the cloud (GitHub)
- File changes are now documented

Basic Git Flow

a set of files

Repository/Repo

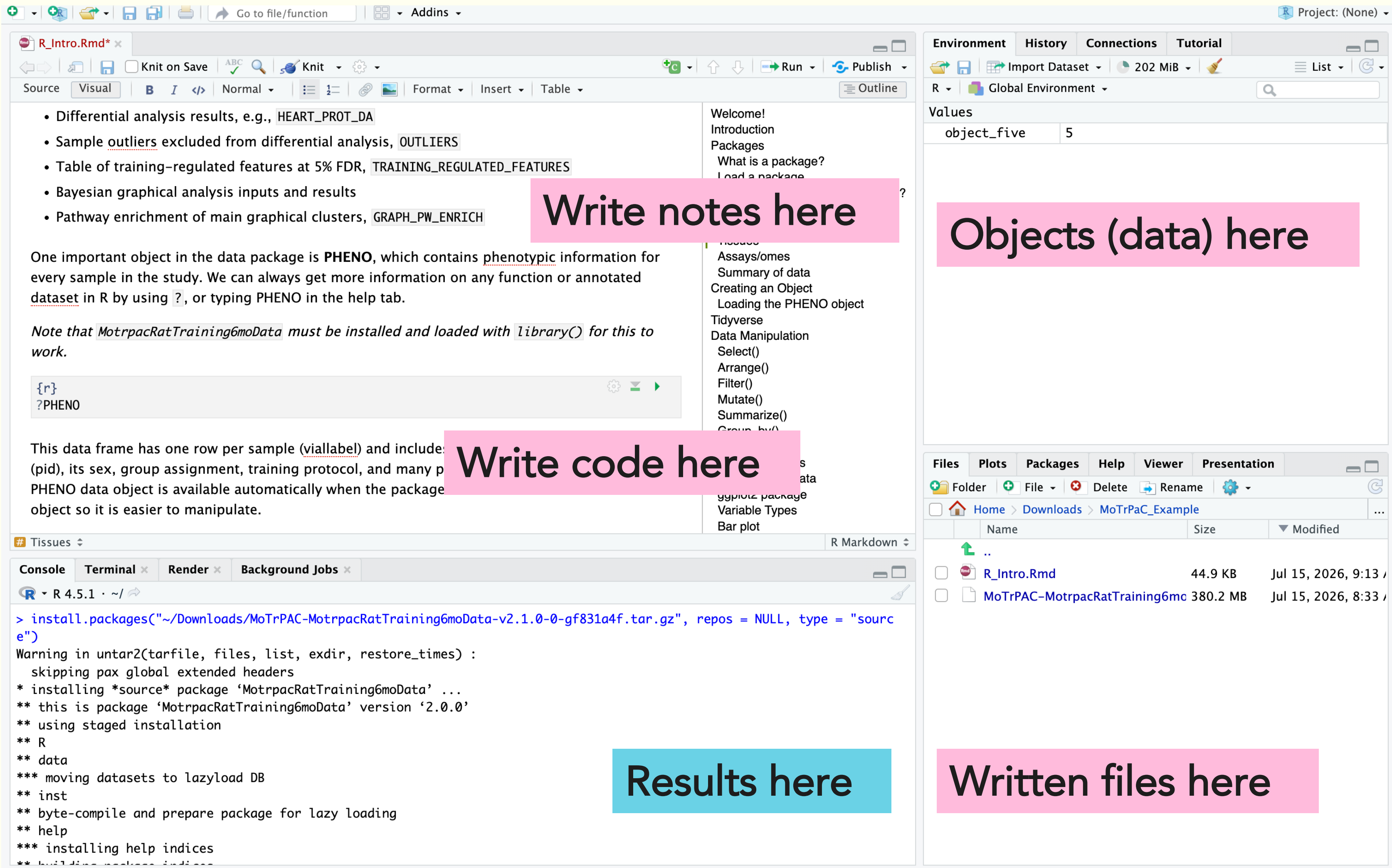
Pull = get files from online
- Get updates to files from online

Diff = add/save
- Add files to repo
- Save any new changes made to files

Commit = describe the change
- Helps convey why the changes were made
- Confirm the changes you want to make

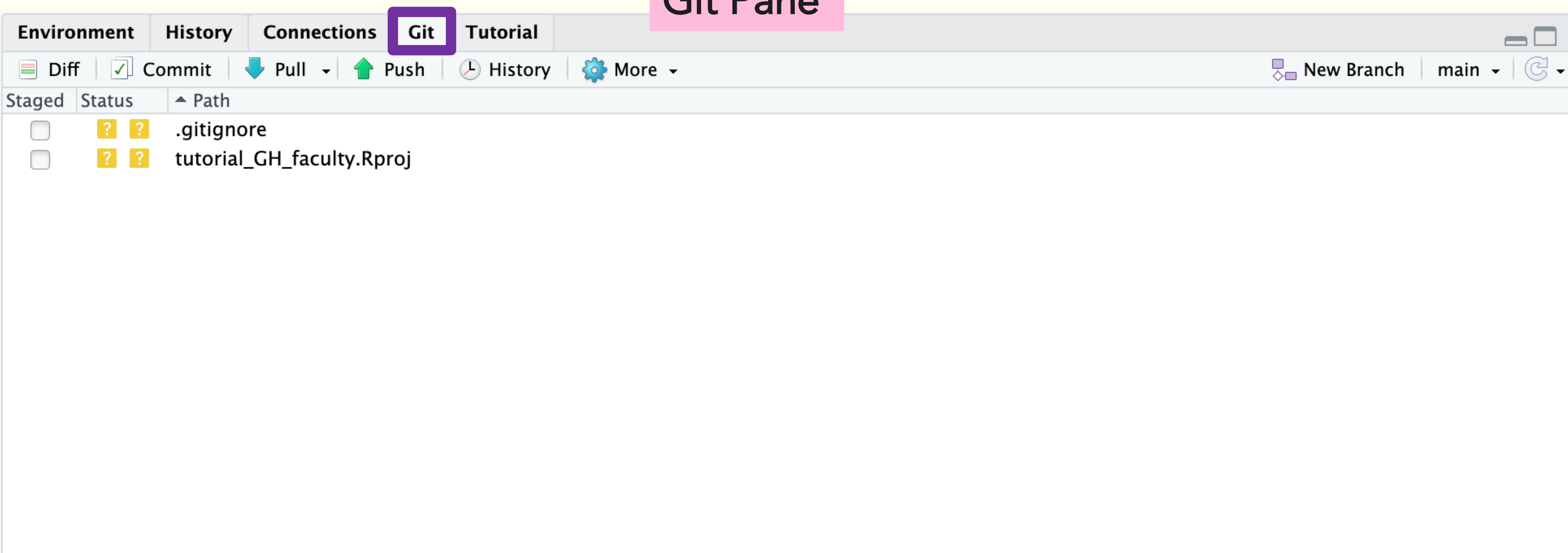
Push = send and store online
- Saves the new files on the cloud (GitHub)
- File changes are now documented

How do we do all of this on RStudio?


Write notes here
Objects (data) here
Write code here
Results here
Written files here
R_Intro.Rmd
Environment
History
Connections
Tutorial
Files
Plots
Packages
Help
Viewer
Presentation
Console
Terminal
Render
Background Jobs
One important object in the data package is PHENO, which contains phenotypic information for every sample in the study. We can always get more information on any function or annotated dataset in R by using ?, or typing PHENO in the help tab.
Note that MotrpacRatTraining6moData must be installed and loaded with library() for this to work.
{r}
?PHENO
object_five 5
R_Intro.Rmd 44.9 KB
MoTrPAC-MotrpacRatTraining6mo 380.2 MB
> install.packages("~/Downloads/MoTrPAC-MotrpacRatTraining6moData-v2.1.0-0-gf831a4f.tar.gz", repos = NULL, type = "source")
Warning in untar2(tarfile, files, list, exdir, restore_times) :
skipping pax global extended headers
* installing *source* package 'MotrpacRatTraining6moData' ...
** this is package 'MotrpacRatTraining6moData' version '2.0.0'
** using staged installation

GitHub Commands via RStudio

Git Pane



GitHub Commands via RStudio

Allows you to commit changes you have made, preparing them to be pushed to the original repo.

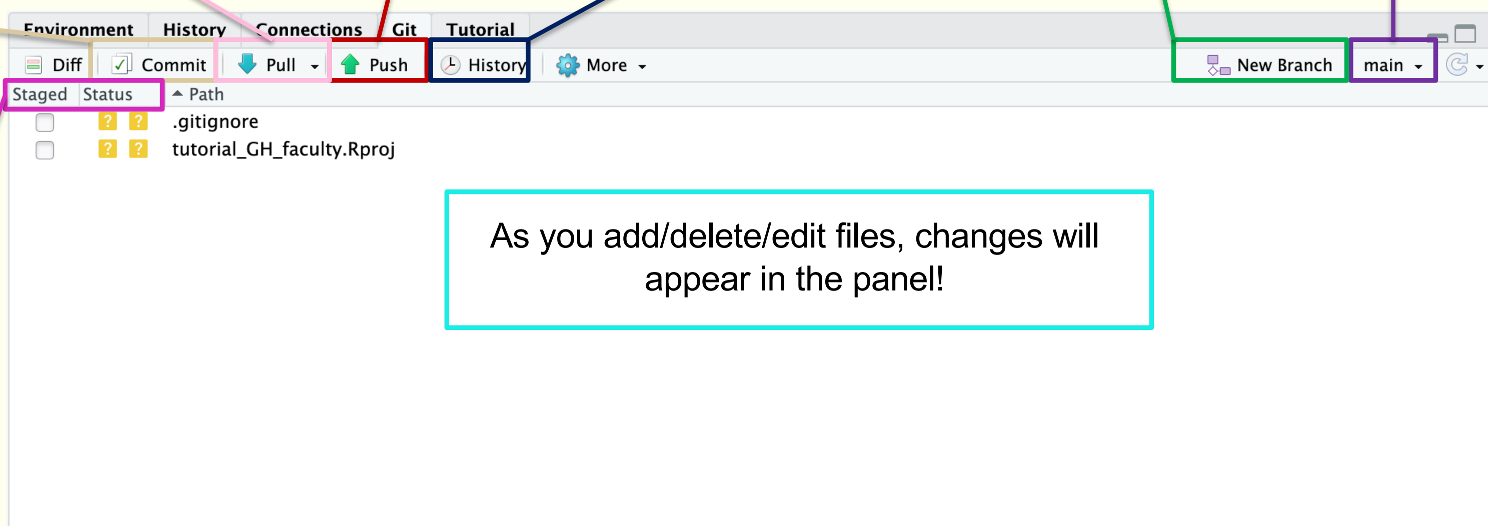
Enables pulling from the original repo. This will sync the online repository with your cloned version.

Here is where you can push commits you have made within your clone to the online repo.

Will show you the history of your project (when files were committed, who was the last person to edit the repo, etc.).

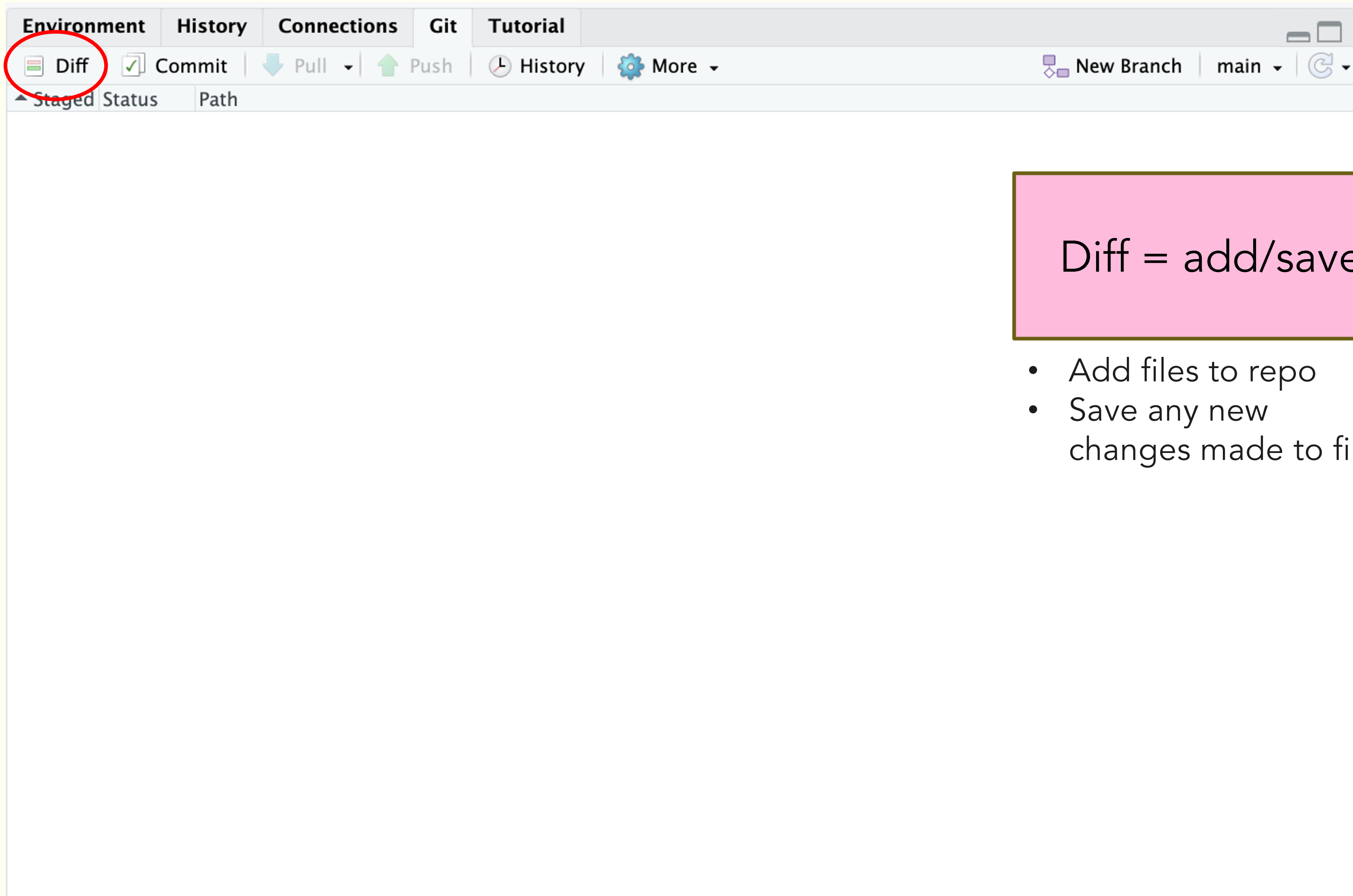
This button allows you to create a separate branch from main. This enables more independent work.

Tells you which branch you are currently working in.



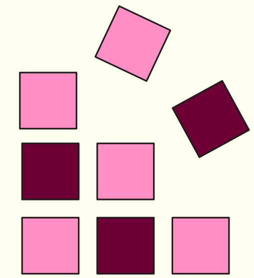
Staged tells you if any changes have been committed. **Status** provides information on each file (has it been deleted, moved, renamed, etc).

As you add/delete/edit files, changes will appear in the panel!



Diff = add/save

- Add files to repo
- Save any new changes made to files



Diff Example

Diff = add/save

- Add files to repo
- Save any new changes made to files

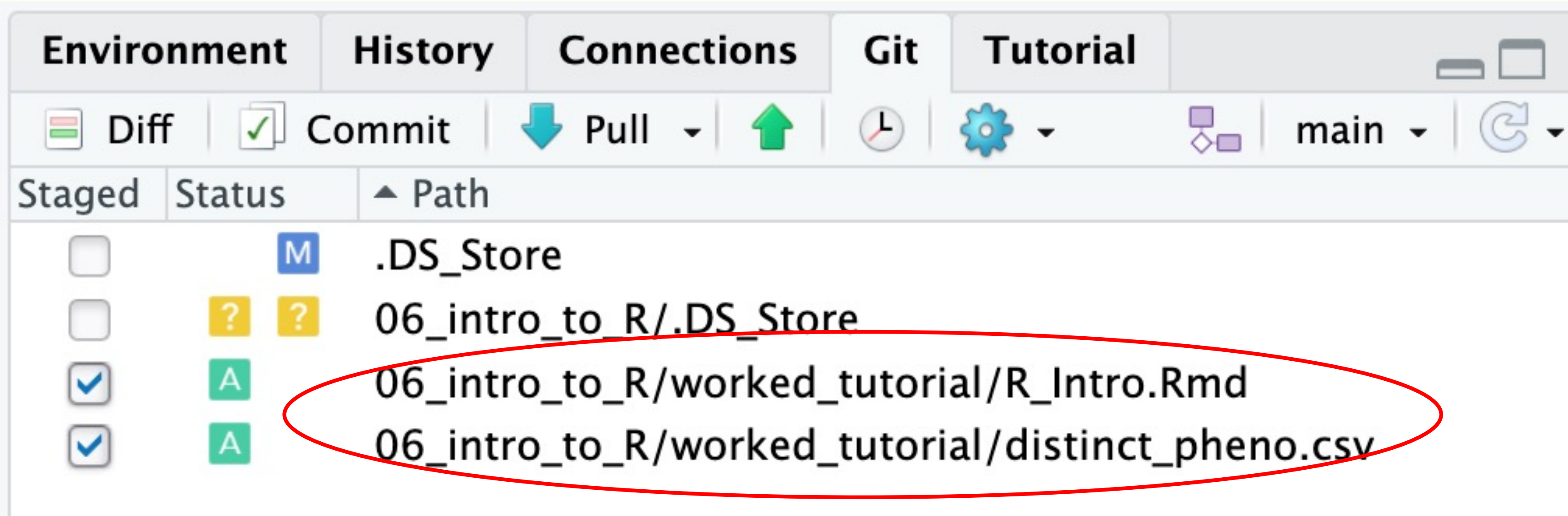
Say we wanted export our filtered dataset from our last workshop into a CSV:

Exporting Data

Let's export some filtered and manipulated data so it can be used in the future!

```
{r}

write.csv(pheno_dist, here("06_intro_to_R/worked_tutorial/distinct_pheno.csv"))
```



We observe that the changed R notebook file and exported CSV shows up in the Git pane

Diff = add/save

- Add files to repo
- Save any new changes made to files

Changes | History | main | Stage | Revert | Ignore | Pull | Push

Staged	Status	Path
☐	M	.DS_Store
☐	? ?	06_intro_to_R/.DS_Store
☐	? ?	06_intro_to_R/worked_tutorial/R_Intro.Rmd
☑	A	06_intro_to_R/worked_tutorial/distinct_pheno.csv

Show ○ Staged ◉ Unstaged Context 5 lines ☐ Ignore Whitespace

```
### Factor Re-leveling

The order of our groups doesn't quite make sense. It would be easier to read the graph if it were ordered from positive to negative rather than alphabetically. If we want to reorder factors, we will use the `fct_relevel` function found within the [forcats](https://forcats.tidyverse.org/) package. We will take the `pheno_exp` object and use `mutate` to overwrite the `group` variable with the `fct_relevel` function on the `group` variable. In this instance, we need to write the exact name and order of each factor by hand. We can check that our reorganization worked by checking the `levels`.

```{r}

pheno_exp <- pheno_exp %>% mutate(group=fct_relevel(group, "control", "1w", "2w", "4w", "8w" ))
levels(pheno_exp$group)
```

Rerun the boxplots to see the controls show up first.

```{r, solution=TRUE}
ggplot(pheno_exp, aes(x = group, y = Gak)) +
  geom_boxplot() +
  facet_wrap(~sex)
```

Exporting Data

Let's export some filtered and manipulated data so it can be used in the future!

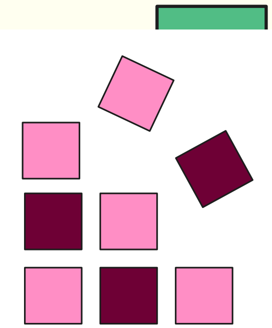
```{r}

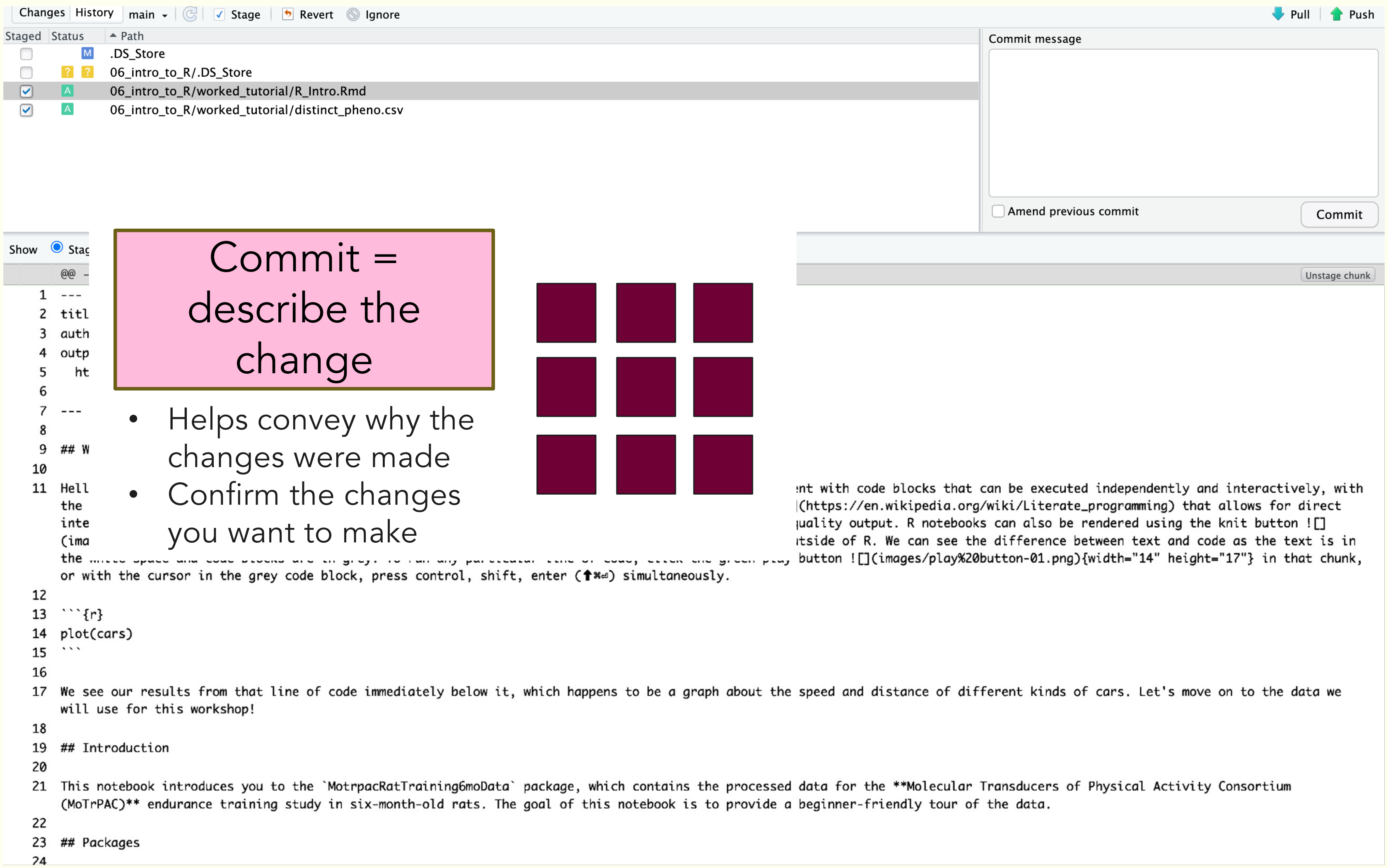
write.csv(pheno_dist, here("06_intro_to_R/worked_tutorial/distinct_pheno.csv"))

```
```

Diff = add/save

- Add files to repo
- Save any new changes made to files



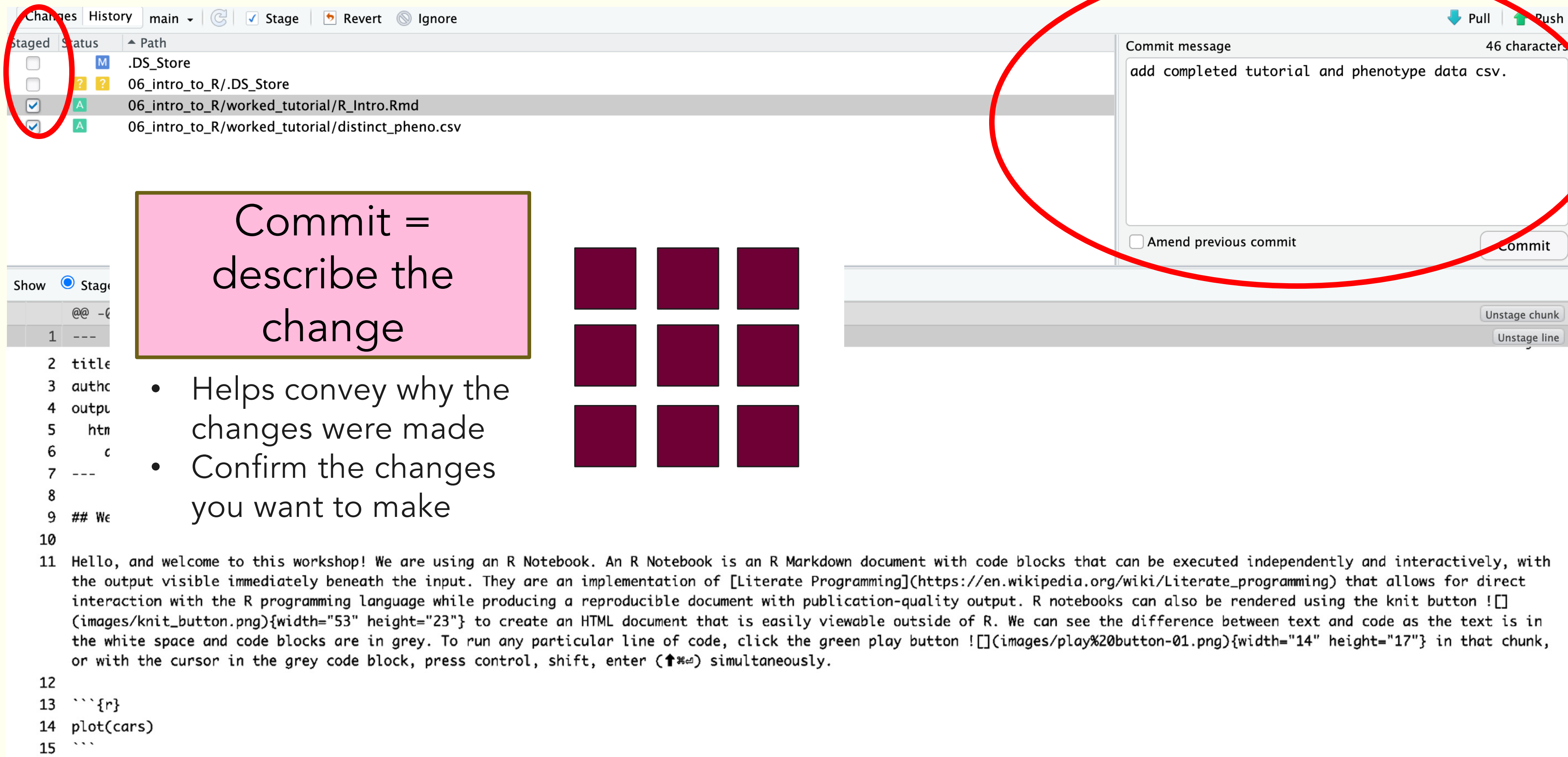


Commit = describe the change

- Helps convey why the changes were made
- Confirm the changes you want to make

Commit = describe the change

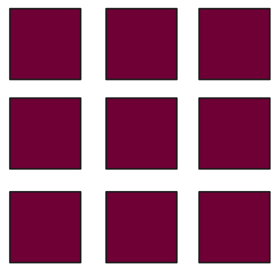
- Helps convey why the changes were made
- Confirm the changes you want to make



Commit message

Commit = describe the change

- Helps convey why the changes were made
- Confirm the changes you want to make

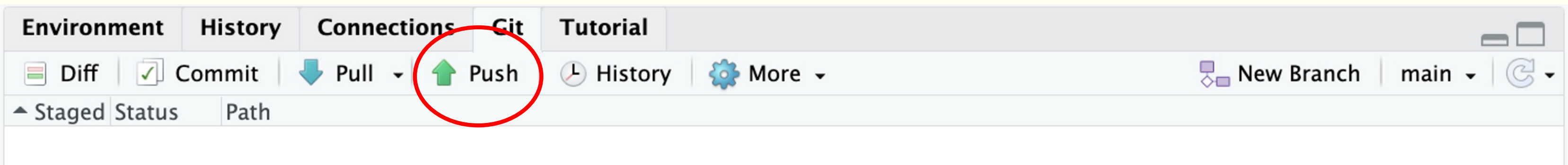


Git Commit

Close

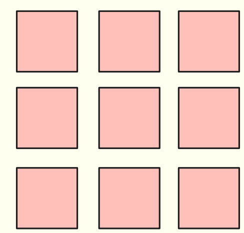
```
>>> /usr/bin/git commit -F /var/folders/9s/mbqgj3bn1z1d6p_hdyj8lbd576jfxy/T/RtmpFSj
[main f71ccfe] add completed tutorial and phenotype data csv.
 2 files changed, 1013 insertions(+)
 create mode 100644 06_intro_to_R/worked_tutorial/distinct_pheno.csv
```

Storing files in the cloud

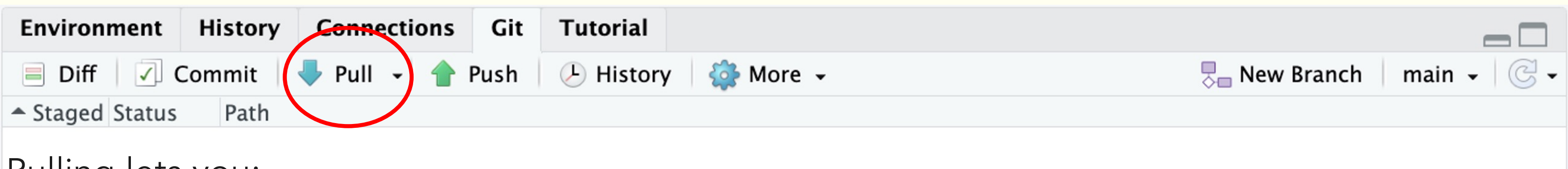


Push = send and store to the cloud

- Saves the new files on the cloud (GitHub)
- File changes are now documented

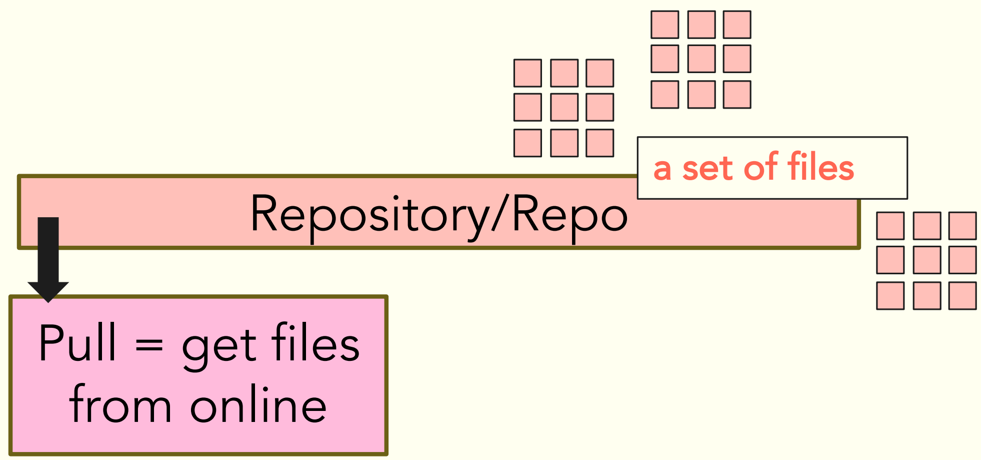


Refreshing your files from the cloud

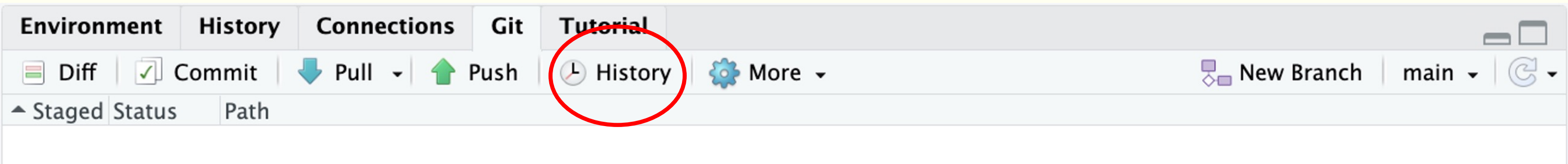


Pulling lets you:

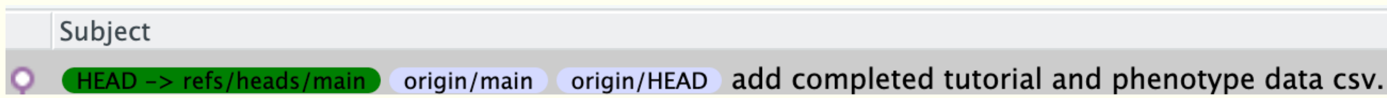
- Access the latest updates to your files
- Pick up your project from any computer from where you left it
- Work with others and bring in any changes your teammates make

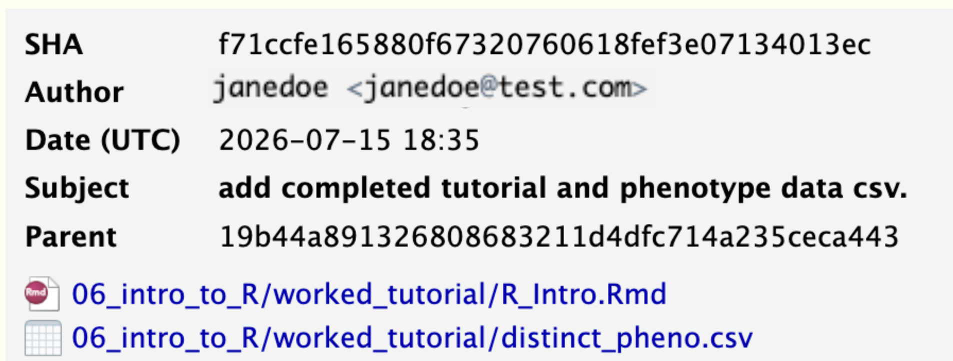


History in Git

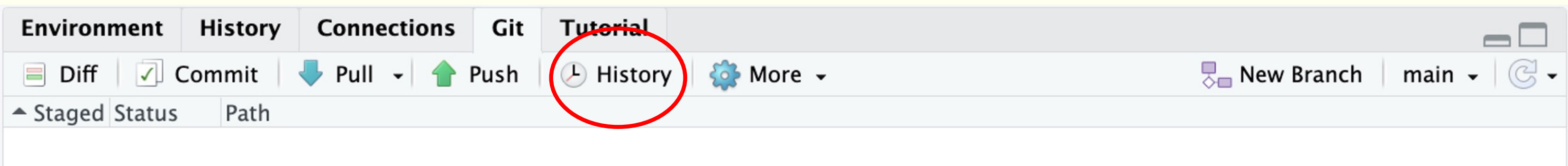


History shows you what changes you've made to your project



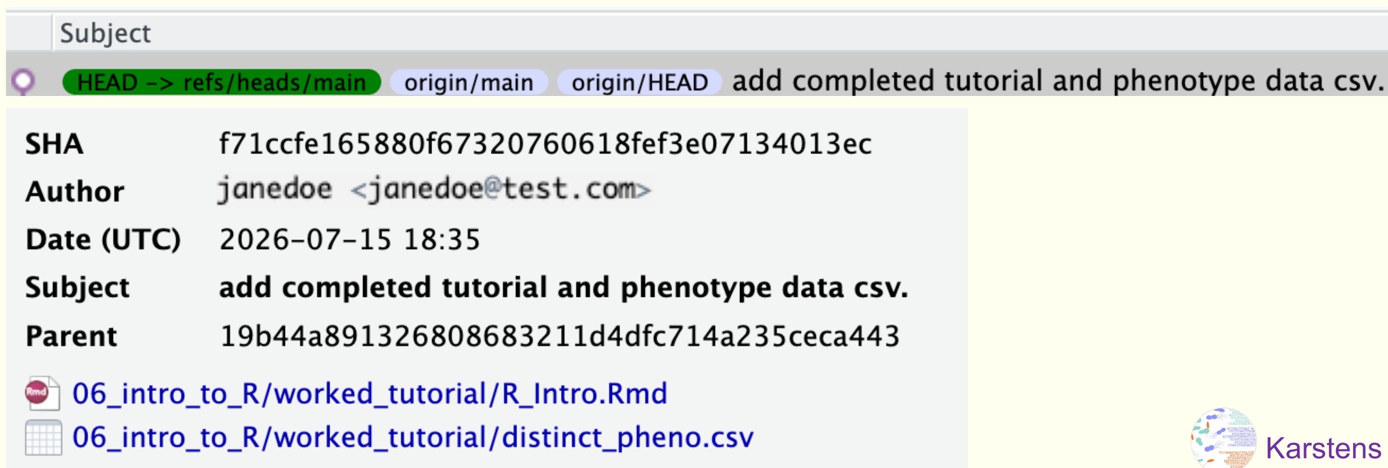


History in Git

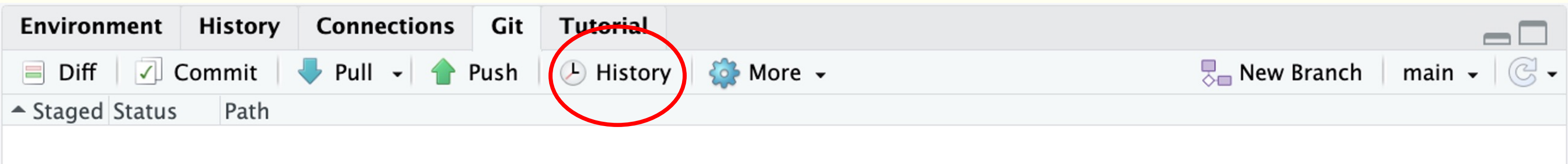


History shows you what changes you've made to your project

- You can also check who made the change and when they were made

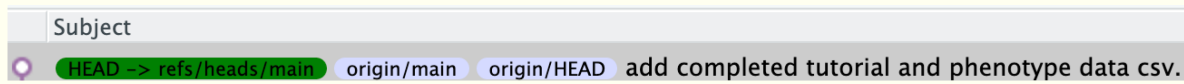


History in Git



History shows you what changes you've made to your project

- You can also check who made the change and when they were made
- You can additionally look at what files were changed, added, or deleted



Getting Ready to Use GitHub

Step 0: Installations

To complete this activity, you must have **Git, R,** and **RStudio** installed on your computing environment.

- R/RStudio Download Instructions (Windows & MacOS)
- Git Install Instructions (Various OS)

You must also have a **GitHub account**!
Github.com → Sign up

→ If you don't have the setup now, don't worry!

Register a GitHub account

Simple Rules:

- **Use your real name** - like "sarah_johnson" or "mikec"
- **Keep it short** - easier to type and remember
- **Make it unique** - so no one else has it
- **Keep it professional** - something you'd be okay with teachers or future bosses seeing
- **Don't use temporary stuff** - avoid your school name or current city
- **Reuse from other places** - if you have LinkedIn or other professional social media accounts, use the same name

Remember: You'll use this username for years, so pick something you'll still like later!

Happy Git with R: Chapter 4

Check Installations

R

Open terminal and type:

```
R --version
```

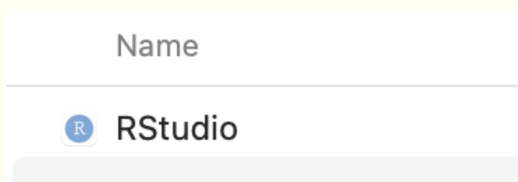
Example output:

```
(base) CP20034:~ janedoe$ R --version
R version 4.5.1 (2025-06-13) -- "Great Square Root"
Copyright (C) 2025 The R Foundation for Statistical
Platform: aarch64-apple-darwin20
```

RStudio

Search for "RStudio" in your applications. You should see the RStudio app available for use.

Example output:



Git

Open terminal app and type:

```
git --version
```

Example output:

```
(base) CP20034:~ janedoe$ git --version
git version 2.39.5 (Apple Git-154)
```

Demo: Github Credentials, GitHub Repo Creation, & Clone Repo in RStudio

Step 1: Introduce yourself to Git

Global configurations apply across repos at the user level.

Two ways:

1. Commands from the terminal

```
git config --global user.name "Jane_Doe"
git config --global user.email "jane@example.com"
git config --global --list
```

***substitute the name with your name and email associated with your GitHub account**

2. Using the *usethis* R package

```
install.packages(c("usethis”, “gitcreds”)

Usethis::use_git_config(user.name = "Jane_Doe", user.email = "jane@example.org")
```

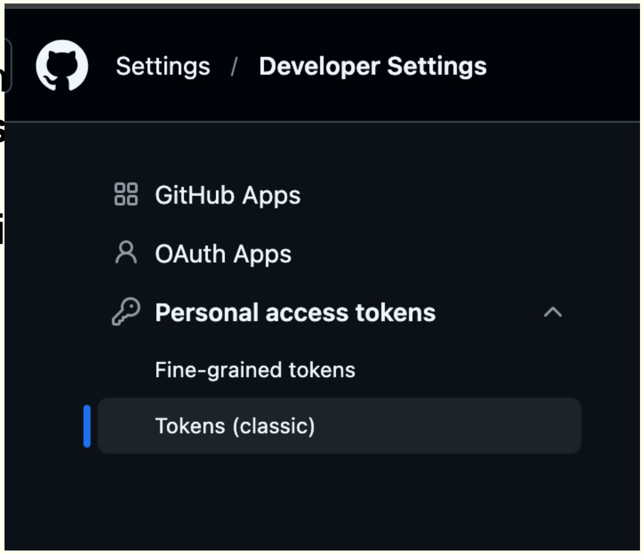
Happy Git with R: Chapter 7

Step 2: Create a Personal Access Token (PAT)

Two ways:

1. On Github:

- Click on your user icon in the upper righthand corner, and th navigate to **Settings/Developer Settings/Personal access tokens/Tokens (classic)**
- Click on Generate new token → Generate new token (classi
- For scopes, select **repo** and **workflow**.
- Tips:
  - Set a time limit for your PAT!
  - Keep it SAFE / PRIVATE!
  - Once you have generated your token, copy, paste, and **save it** immediately!



2. Using the *usethis* R package

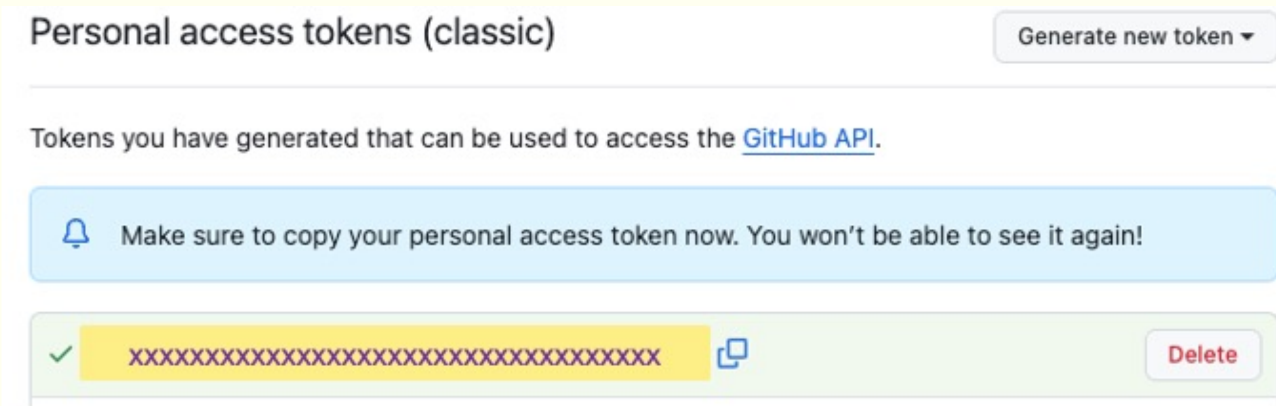
```
usethis::create_github_token()
gitcreds::gitcreds_set()
```

More details: usethis.r-lib.org/articles/git-credentials.html

Step 2: Create a Personal Access Token (PAT)

Using the *usethis* R package

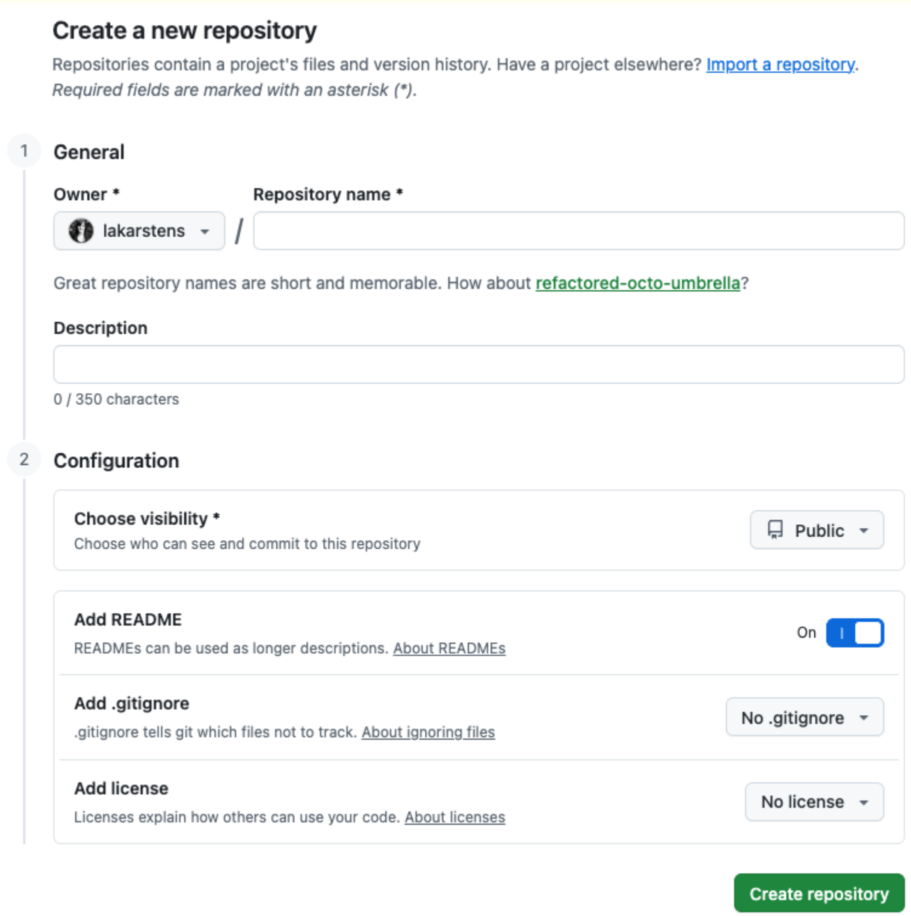
```
gitcreds::gitcreds_set()
```



- Tips:
  - Set a time limit for your PAT!
  - Keep it SAFE / PRIVATE!
  - Once you have generated your token, copy, paste, and **save it** immediately!

More details: usethis.r-lib.org/articles/git-credentials.html

Step 3: Create Repo on Github

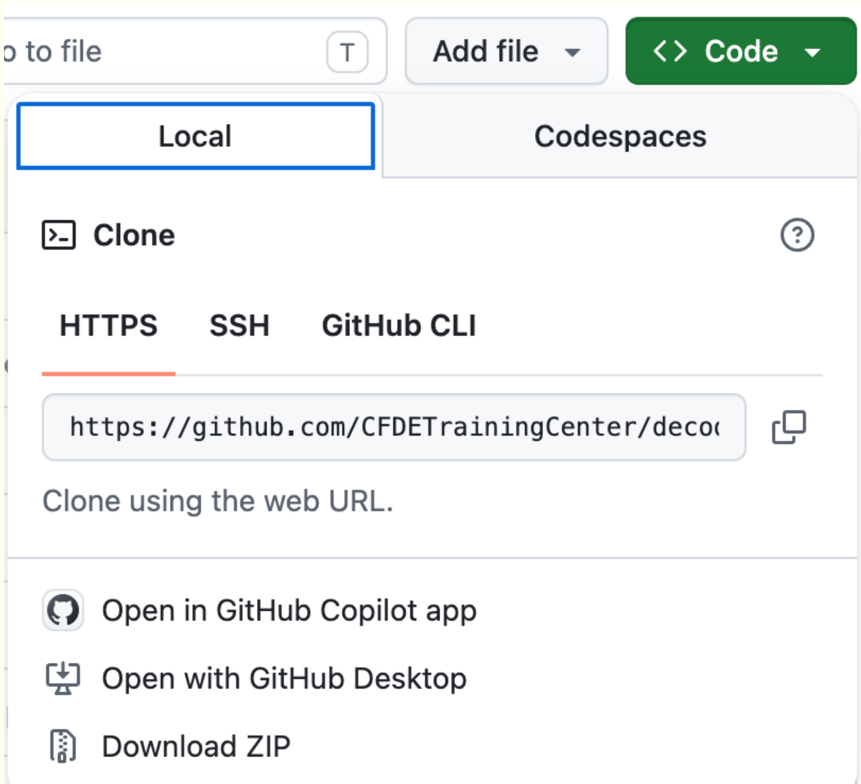


Simple Rules:

- Make the name relevant to the task at hand
- We suggest something like: decode_institute_git_<your_name>

Remember: You'll want to name this something clear enough that you can find your work in the future.

Step 4: Copy Repo URL

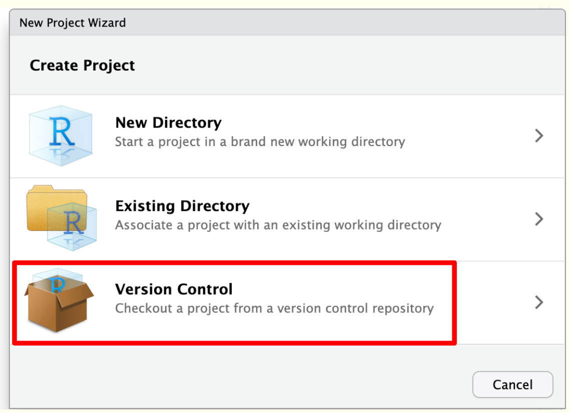


- Navigate to your Repo's home page
- Select Code
- Copy the HTTPS link to the repository.

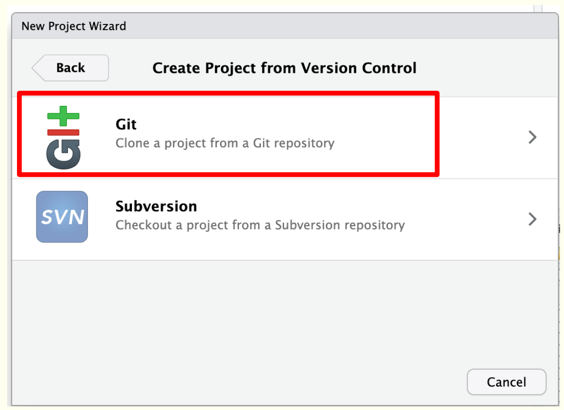
Karstens Lab

Step 5: Clone Repo

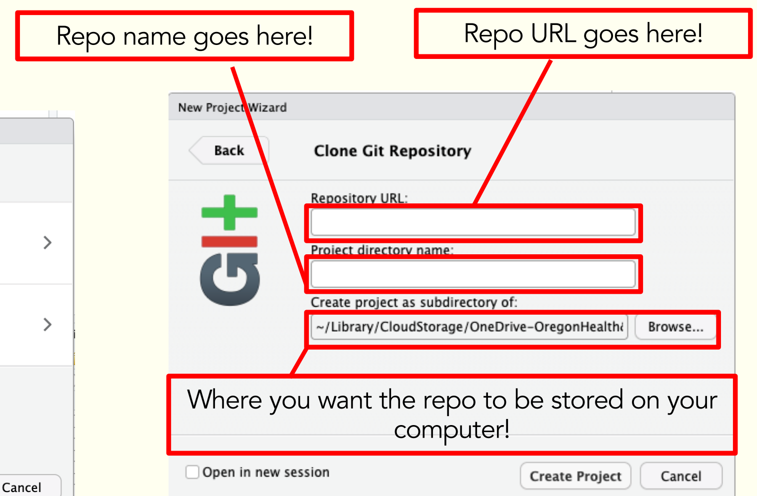
- Now that you have created a repo on GitHub, the next step is to clone (copy) that repo into RStudio.
- Begin in RStudio by clicking **File → New Project.**
- From there, follow the steps listed below:



Select Version Control



Click on Git



Fill in the information about the repository.

Note: If prompted for a password, **ENTER YOUR PAT,** instead of your account password.

Demo: Diff/Save > Commit > Push

Step 6: Create Code


````
```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE)
```

## Example

```{r}

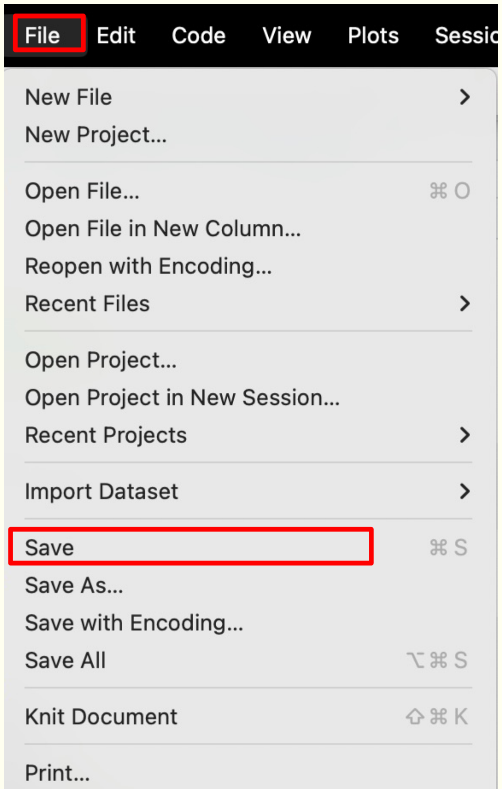
library(ggplot2)

data(mtcars)

 ggplot(mtcars, aes(x=factor(cyl))) +
     geom_bar() +
     xlab("Number of Cylinders") +
     ylab("Frequency")
```
````

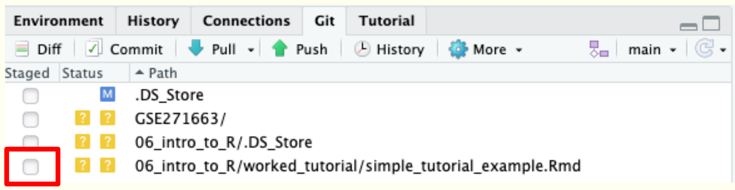
- We are going to create a simple visualization using the mtcars dataset.
- This is a simple dataset that comes preloaded with R.
- The code on the left illustrates what the final product will look like.

Step 7: Save

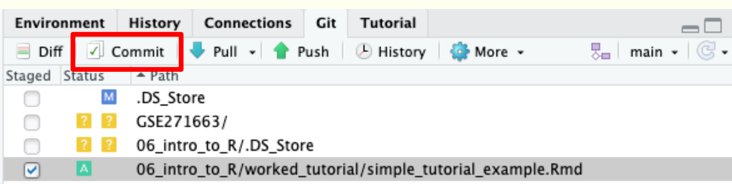


- Once you have completed the plot edits, it is time to save and rename your file.
- Click on File → Save
- Save the file <your_name>_cars_example.Rmd

Step 8: Stage + Commit

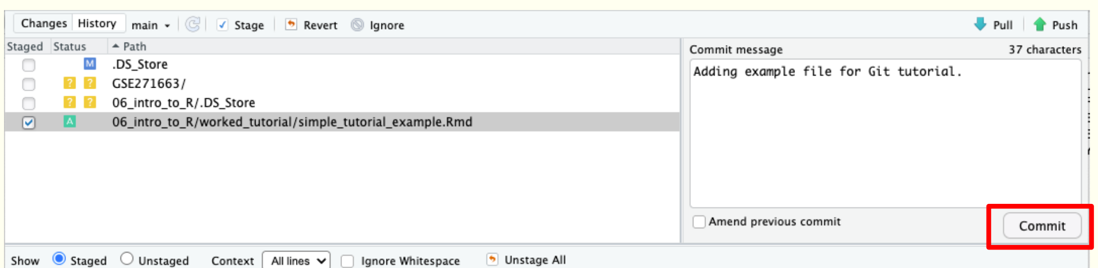


- To stage the file, click on the checkbox next to your file's name in the RStudio Git panel.
- Click on Commit
- Write a commit message explaining what changes you have made to your repo.
- When you're done, click on the <u>Commit</u> button

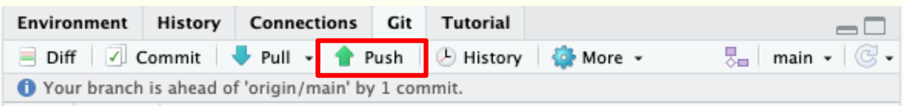


Commit message tips:

- Keep it short, to avoid cluttering the repo
- Summarize changes so they are clear to you when you look back at your work.



Step 9: Push



- To complete the workflow, select the Push icon.
- This will sync the changes you made in RStudio to the original repo.
- An example of the type of message you should see in RStudio is shown on the left.

- To make sure everything worked, go back to your repo and see if your file is now visible!

Additional steps

This is just the start!

Here's a few more common uses:

- Pull changes that were made by someone else

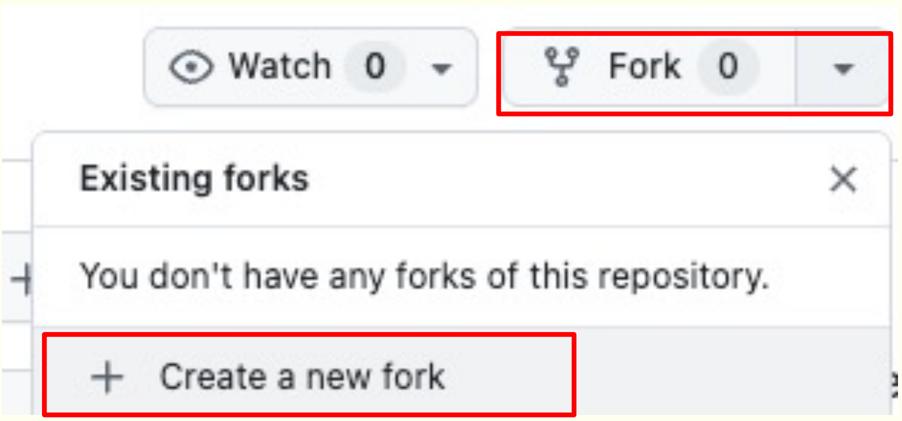
Ands lots of functionality we didn't cover:

- Add a collaborator
- Branching and pull requests
- Code review and feedback
- Issue tracking

Your Turn!

Fork > Diff/Save > Commit > Push

Additional Activity: Fork a Repo



- Navigate to the github.com/jdkudrick/decode-institute-2026-git-example
- Click on Fork → Create a new fork
- This will take you to a new page (similar to when you created your repository) where you will create your forked repo.

You can keep the name the same or change it – it's up to you!

If you previously created a fork – pull to update it!

Additional Activity: Modify a File

Example Visualization

Below, we create a simple histogram using `ggplot`.

```
{r example_viz}

ggplot(pheno_select, aes(x = vo_max1)) + geom_histogram()
```

Your Turn!

Building off of the plot above, complete the following:

- Facet by sex of the mouse (call `facet_wrap()`)
- Fill in bars based on sex (`fill = sex`)

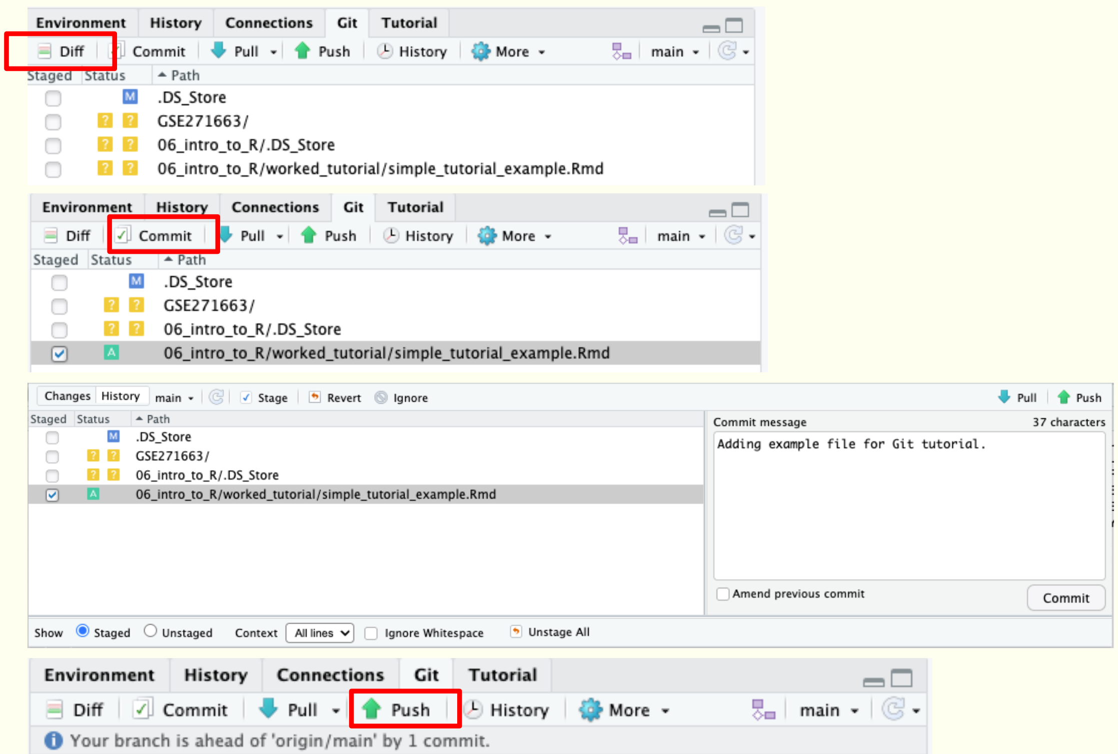
```
{r student_viz}

ggplot(pheno_select, aes(x = vo_max1)) +
  geom_histogram()
```

- Clone your forked repo into RStudio, following the same steps covered on previous slides.
- Open small_tutorial_example.Rmd
- The final code chunk (titled Your Turn!) instructs you to make edits to a plot.

This plot is visualizing some MoTrPAC data that was used during previous workshops

Additional Activity: Commit and Push



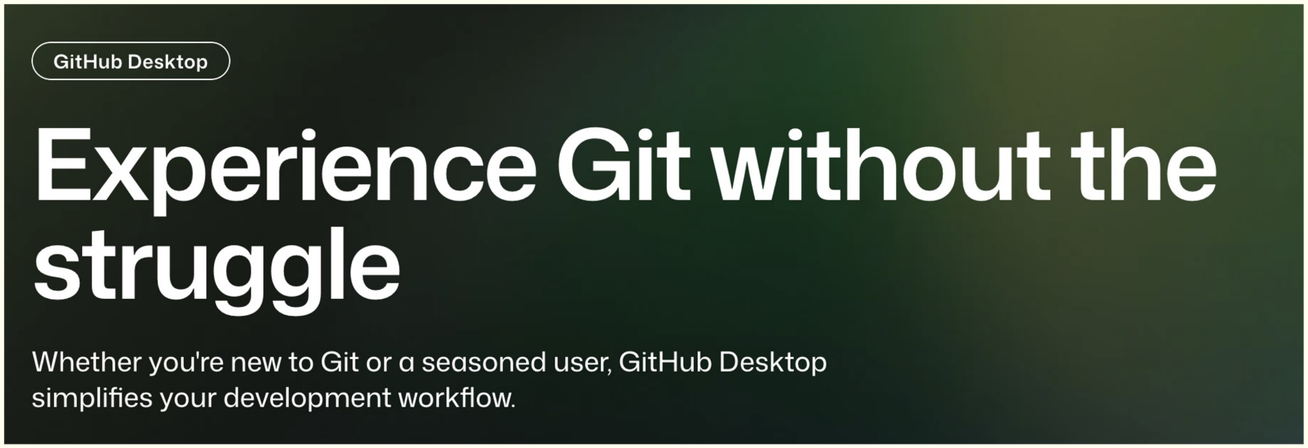
- *Diff, Commit, and Push to your forked repo on github!*
- *Review on github once complete!*

Review of Key Terms

Repository (Repo)	All files and folders associated with a given project. These live online on GitHub.
Clone	Creating a copy of an online repository on your local device, allowing you to make edits locally.
Fork	Creating an online copy of a repository that you can edit without impacting the original repo.
Diff	Adding or modifying files to be tracked with git
Commit	Telling git to track the changes in a particular file
Push	Merging edits from your local device into the online repository.
Pull	Syncing changes from the online repository with your local version.
Issue	A way to report problems to the owner of a repository. Particularly helpful if using open-source packages.
Branch	Creating a separate channel where you can complete edits without modifying the main code (on the main branch). This allows for multiple people to work on the same projects at the same time, without overwriting one another.

Github Desktop

Github Desktop



- Gives you a simple interface to replicate the Git flow without commands

https://github.com/apps/desktop

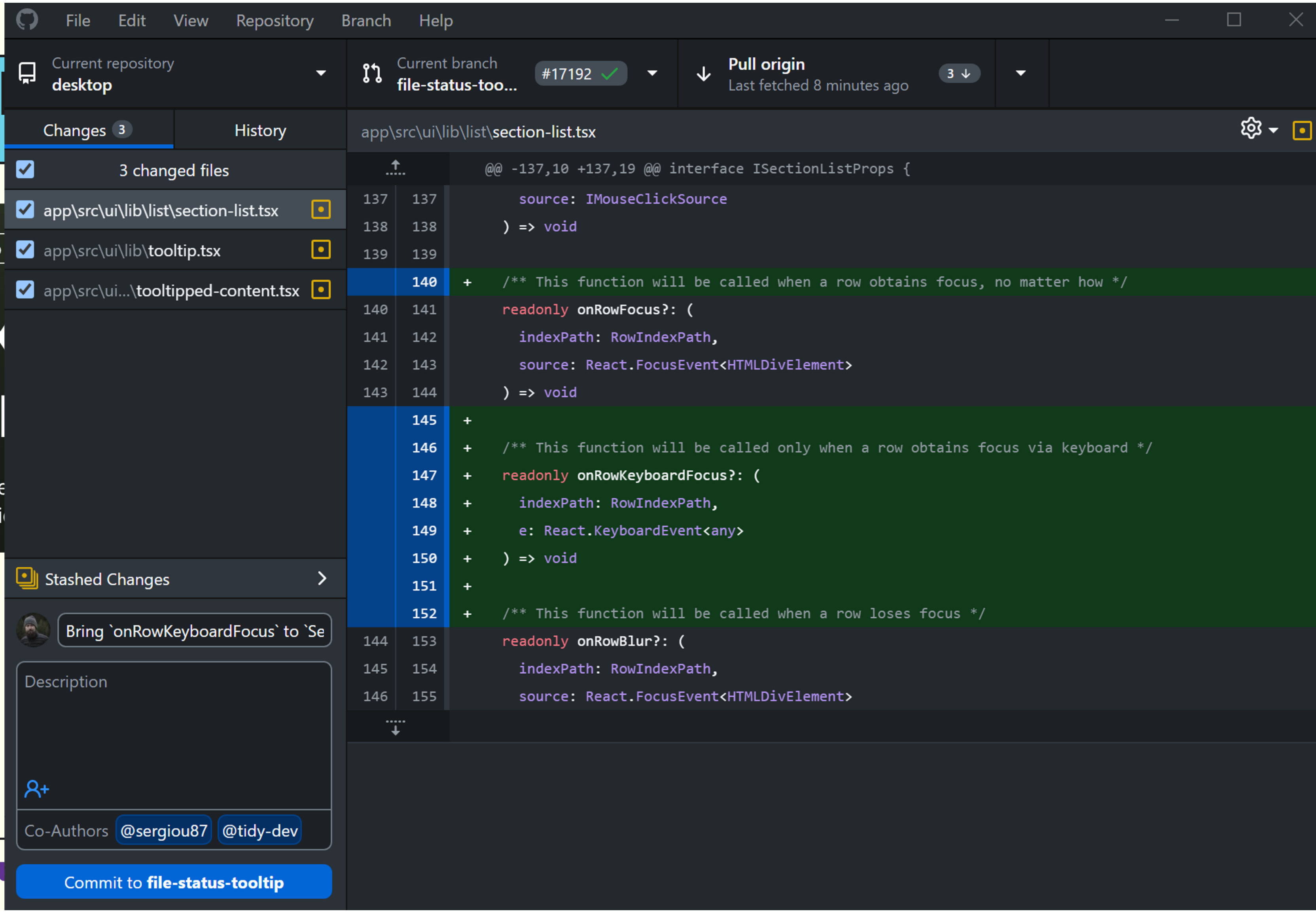
Git

GitHub

Ex
st

Whethe
simplifi

rface to
ithout

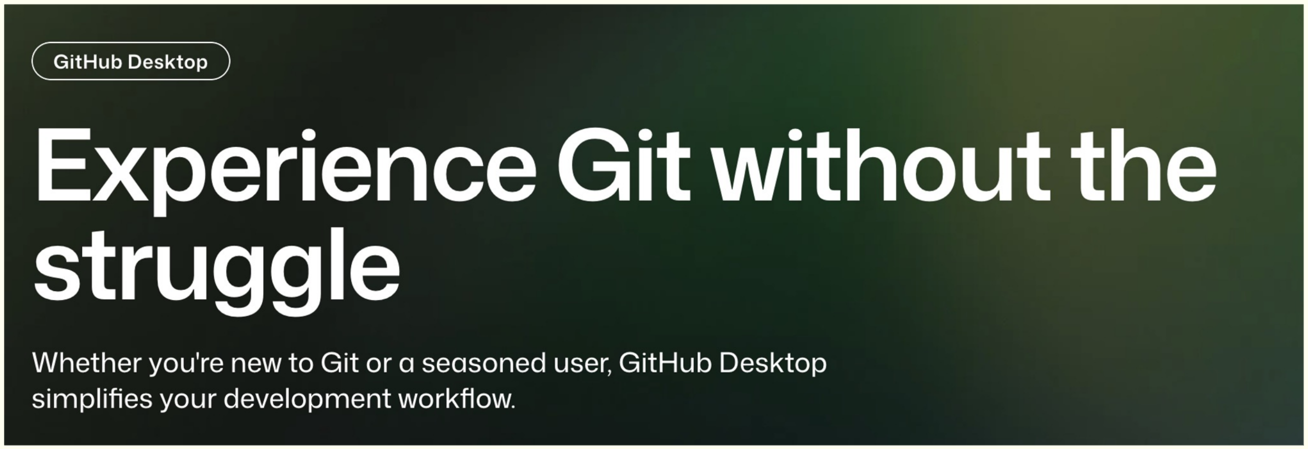


https://g

BioDat

Karstens Lab

Github Desktop



- Gives you a simple interface to replicate the Git flow without commands

For tutorials and how to use GitHub Desktop:
https://docs.github.com/en/desktop

https://github.com/apps/desktop

Additional Resources

1. Happy Git with R: https://happygitwithr.com/
2. GitHub's Git tutorial: https://docs.github.com/en/get-started/git-basics/set-up-git
3. Open and Reproducible Science in R: https://oxford-ihtm.io/ihtm-handbook/

Q&A

Thank you for attending!